Image_Captioning

1. CNN-RNN and CNN-LSTM model for Image Captioning
    1. CNN as an encoder is used to learn features in images.
    2. LSTM as a decoder generates a text sequence describing image features.
    3. Pretrained CNN like VGG-16 or ResNet after removing the final FC classification layer acts as a feature extractor that compresses the image information into a smaller feature vector x.
    4. Input weights, $W_{\text{embed}} : shape : (V, D)$, where V is the vocabulary size, captions, $\text{captions} : shape : (N, T)$ containing integers $0, \ldots, V-1$ for V vocabularies. W_{embed} is indexed by captions to obtain word embeddings, x as input to LSTM which is of shape (N, T, D).
    5. $x : (N, T) \rightarrow W_{\text{embed}} : (V, D) \rightarrow x_{\text{embed}} : (N, T, D)$
    6. example: $\text{captions} = \begin{bmatrix} 2 & 0 & 3 & 5 & 1 & 7 \\ 6 & 0 & 1 & \ldots & \ldots & \ldots \\ 2 & 2 & 3 & \ldots & \ldots & \ldots \end{bmatrix}$, $W = \begin{bmatrix} 0.1 & 0.8 & 0.025 & 0.11 \\ 0.55 & 0.2 & 0.35 & \ldots \\ 0.35 & 0.07 & 0.5 & \ldots \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$
    7. example, one-hot encoding:

```
# one-hot encoding is a special case when V=D, where
# the embedding is a Identity matrix of shape (V, V)
N, T, V, D = 3, 4, 5, 5

# N: batch_size
# V: vocab_dim
# T: sequence_length
caption = np.random.randint(0, V, (N, T))  # words in a vocabulary
embedding = np.eye(V)
x = embedding[caption]

print(x)
print(caption)
```

```
[[[0. 0. 0. 1. 0.]
  [0. 0. 0. 1. 0.]
  [0. 0. 1. 0. 0.]
  [0. 0. 1. 0. 0.]]

 [[0. 0. 0. 0. 1.]
  [0. 0. 0. 0. 1.]
  [0. 0. 1. 0. 0.]
  [0. 0. 1. 0. 0.]]

 [[0. 0. 0. 1. 0.]
  [0. 0. 1. 0. 0.]
  [0. 0. 0. 1. 0.]
  [0. 0. 0. 0. 1.]]]
[[3 3 2 2]
 [4 4 2 2]
 [3 2 3 4]]
```

    8. word_embedding example taken from CS231 in Neural_Networks. RNN and LSTM layers of CNN-RNN and CNN-LSTM for image captioning available in Neural_Networks.
    9. Code: from rnn_layers.py

```
def word_embedding_forward(x, W):
    """
    Forward pass for word embeddings. We operate on minibatches of size N where
    each sequence has length T. We assume a vocabulary of V words, assigning each
    to a vector of dimension D.

    Inputs:
    - x: Integer array of shape (N, T) giving indices of words. Each element idx
      of x muxt be in the range 0 <= idx < V.
    - W: Weight matrix of shape (V, D) giving word vectors for all words.

    Returns a tuple of:
    - out: Array of shape (N, T, D) giving word vectors for all input words.
    - cache: Values needed for the backward pass
    """
    out, cache = None, None
    ##############################################################################
    # TODO: Implement the forward pass for word embeddings.                      #
    #                                                                            #
    # HINT: This can be done in one line using NumPy's array indexing.          #
    ##############################################################################
    out = np.array([W[i, :] for i in x])
    cache = x, W
    ##############################################################################
    #                               END OF YOUR CODE                             #
```

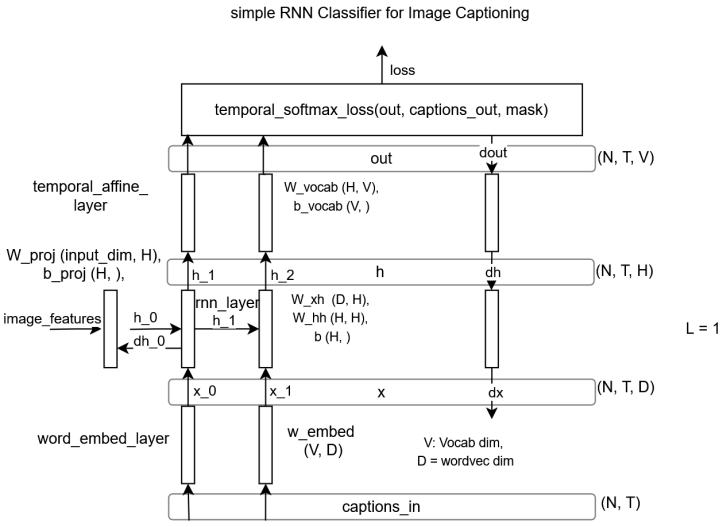
```
    ##############################################################################
    return out, cache


def word_embedding_backward(dout, cache):
    """
    Backward pass for word embeddings. We cannot back-propagate into the words
    since they are integers, so we only return gradient for the word embedding
    matrix.

    HINT: Look up the function np.add.at

    Inputs:
    - dout: Upstream gradients of shape (N, T, D)
    - cache: Values from the forward pass

    Returns:
    - dW: Gradient of word embedding matrix, of shape (V, D).
    """
    dW = None
    ##############################################################################
    # TODO: Implement the backward pass for word embeddings.                     #
    #                                                                            #
    # Note that Words can appear more than once in a sequence.                   #
    # HINT: Look up the function np.add.at                                       #
    ##############################################################################
    x, W = cache
    dW = np.zeros(W.shape)
    np.add.at(dW, x, dout)
    ##############################################################################
    #                                END OF YOUR CODE                            #
    ##############################################################################
    return dW
```



10. Code: classifier/rnn.py

```
from builtins import range
from builtins import object
import numpy as np

from cs231n.layers import *
from cs231n.rnn_layers import *


class CaptioningRNN(object):
    """
    A CaptioningRNN produces captions from image features using a recurrent
    neural network.

    The RNN receives input vectors of size D, has a vocab size of V, works on
    sequences of length T, has an RNN hidden dimension of H, uses word vectors
    of dimension W, and operates on minibatches of size N.

    Note that we don't use any regularization for the CaptioningRNN.
    """

    def __init__(self, word_to_idx, input_dim=512, wordvec_dim=128,
                 hidden_dim=128, cell_type='rnn', dtype=np.float32):
        """
        Construct a new CaptioningRNN instance.

        Inputs:
        - word_to_idx: A dictionary giving the vocabulary. It contains V entries,
          and maps each string to a unique integer in the range [0, V).
        - input_dim: Dimension D of input image feature vectors.
        - wordvec_dim: Dimension W of word vectors.
```

```
        - hidden_dim: Dimension H for the hidden state of the RNN.
        - cell_type: What type of RNN to use; either 'rnn' or 'lstm'.
        - dtype: numpy datatype to use; use float32 for training and float64 for
          numeric gradient checking.
        """
        if cell_type not in {'rnn', 'lstm'}:
            raise ValueError('Invalid cell_type "%s"' % cell_type)

        self.cell_type = cell_type
        self.dtype = dtype
        self.word_to_idx = word_to_idx
        self.idx_to_word = {i: w for w, i in word_to_idx.items()}
        self.params = {}

        vocab_size = len(word_to_idx)

        self._null = word_to_idx['<NULL>']
        self._start = word_to_idx.get('<START>', None)
        self._end = word_to_idx.get('<END>', None)

        # Initialize word vectors
        self.params['W_embed'] = np.random.randn(vocab_size, wordvec_dim)
        self.params['W_embed'] /= 100

        # Initialize CNN -> hidden state projection parameters
        self.params['W_proj'] = np.random.randn(input_dim, hidden_dim)
        self.params['W_proj'] /= np.sqrt(input_dim)
        self.params['b_proj'] = np.zeros(hidden_dim)

        # Initialize parameters for the RNN
        dim_mul = {'lstm': 4, 'rnn': 1}[cell_type]
        self.params['Wx'] = np.random.randn(wordvec_dim, dim_mul * hidden_dim)
        self.params['Wx'] /= np.sqrt(wordvec_dim)
        self.params['Wh'] = np.random.randn(hidden_dim, dim_mul * hidden_dim)
        self.params['Wh'] /= np.sqrt(hidden_dim)
        self.params['b'] = np.zeros(dim_mul * hidden_dim)

        # Initialize output to vocab weights
        self.params['W_vocab'] = np.random.randn(hidden_dim, vocab_size)
        self.params['W_vocab'] /= np.sqrt(hidden_dim)
        self.params['b_vocab'] = np.zeros(vocab_size)

        # Cast parameters to correct dtype
        for k, v in self.params.items():
            self.params[k] = v.astype(self.dtype)


    def loss(self, features, captions):
        """
        Compute training-time loss for the RNN. We input image features and
        ground-truth captions for those images, and use an RNN (or LSTM) to compute
        loss and gradients on all parameters.

        Inputs:
        - features: Input image features, of shape (N, D)
        - captions: Ground-truth captions; an integer array of shape (N, T) where
          each element is in the range 0 <= y[i, t] < V

        Returns a tuple of:
        - loss: Scalar loss
        - grads: Dictionary of gradients parallel to self.params
        """
        # Cut captions into two pieces: captions_in has everything but the last word
        # and will be input to the RNN; captions_out has everything but the first
        # word and this is what we will expect the RNN to generate. These are offset
        # by one relative to each other because the RNN should produce word (t+1)
        # after receiving word t. The first element of captions_in will be the START
        # token, and the first element of captions_out will be the first word.

        # Captions: shape: (N, T)
        captions_in = captions[:, :-1]  # (:, :T-1)
        captions_out = captions[:, 1:]  # (:, 1:T)

        # You'll need this
        mask = (captions_out != self._null)

        # Weight and bias for the affine transform from image features to initial
        # hidden state
        W_proj, b_proj = self.params['W_proj'], self.params['b_proj']

        # Word embedding matrix
        W_embed = self.params['W_embed']

        # Input-to-hidden, hidden-to-hidden, and biases for the RNN
        Wx, Wh, b = self.params['Wx'], self.params['Wh'], self.params['b']

        # Weight and bias for the hidden-to-vocab transformation.
        W_vocab, b_vocab = self.params['W_vocab'], self.params['b_vocab']

        loss, grads = 0.0, {}
```

```
        ############################################################################
        # TODO: Implement the forward and backward passes for the CaptioningRNN.   #
        # In the forward pass you will need to do the following:                   #
        # (1) Use an affine transformation to compute the initial hidden state     #
        #     from the image features. This should produce an array of shape (N, H)#
        # (2) Use a word embedding layer to transform the words in captions_in     #
        #     from indices to vectors, giving an array of shape (N, T, W).         #
        # (3) Use either a vanilla RNN or LSTM (depending on self.cell_type) to    #
        #     process the sequence of input word vectors and produce hidden state  #
        #     vectors for all timesteps, producing an array of shape (N, T, H).    #
        # (4) Use a (temporal) affine transformation to compute scores over the    #
        #     vocabulary at every timestep using the hidden states, giving an      #
        #     array of shape (N, T, V).                                            #
        # (5) Use (temporal) softmax to compute loss using captions_out, ignoring  #
        #     the points where the output word is <NULL> using the mask above.     #
        #                                                                          #
        # In the backward pass you will need to compute the gradient of the loss   #
        # with respect to all model parameters. Use the loss and grads variables   #
        # defined above to store loss and gradients; grads[k] should give the      #
        # gradients for self.params[k].                                            #
        ############################################################################
        if self.cell_type == 'rnn':
          # Perform image feature projection
          h0 = features @ W_proj + b_proj
          # Word embedding layer
          x, x_cache = word_embedding_forward(captions_in, W_embed)
          h, h_cache = rnn_forward(x, h0, Wx, Wh, b)
          out, out_cache = temporal_affine_forward(h, W_vocab, b_vocab)

          loss, dout = temporal_softmax_loss(out, captions_out, mask)

          dh, grads['W_vocab'], grads['b_vocab'] = temporal_affine_backward(dout, out_cache)
          dx, dh0, grads['Wx'], grads['Wh'], grads['b'] = rnn_backward(dh, h_cache)
          grads['W_embed'] = word_embedding_backward(dx, x_cache)
          grads['W_proj'] = (dh0.T @ features).T
          grads['b_proj'] = dh0.sum(axis=0)
        elif self.cell_type == 'lstm':
          h0 = features @ W_proj + b_proj
          # Word embedding layer
          x, x_cache = word_embedding_forward(captions_in, W_embed)
          h, h_cache = lstm_forward(x, h0, Wx, Wh, b)
          out, out_cache = temporal_affine_forward(h, W_vocab, b_vocab)

          loss, dout = temporal_softmax_loss(out, captions_out, mask)

          dh, grads['W_vocab'], grads['b_vocab'] = temporal_affine_backward(dout, out_cache)
          dx, dh0, grads['Wx'], grads['Wh'], grads['b'] = lstm_backward(dh, h_cache)
          grads['W_embed'] = word_embedding_backward(dx, x_cache)
          grads['W_proj'] = (dh0.T @ features).T
          grads['b_proj'] = dh0.sum(axis=0)

        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


    def sample(self, features, max_length=30):
        """
        Run a test-time forward pass for the model, sampling captions for input
        feature vectors.

        At each timestep, we embed the current word, pass it and the previous hidden
        state to the RNN to get the next hidden state, use the hidden state to get
        scores for all vocab words, and choose the word with the highest score as
        the next word. The initial hidden state is computed by applying an affine
        transform to the input image features, and the initial word is the <START>
        token.

        For LSTMs you will also have to keep track of the cell state; in that case
        the initial cell state should be zero.

        Inputs:
        - features: Array of input image features of shape (N, D).
        - max_length: Maximum length T of generated captions.

        Returns:
        - captions: Array of shape (N, max_length) giving sampled captions,
          where each element is an integer in the range [0, V). The first element
          of captions should be the first sampled word, not the <START> token.
        """
        N = features.shape[0]
        captions = self._null * np.ones((N, max_length), dtype=np.int32)

        # Unpack parameters
        W_proj, b_proj = self.params['W_proj'], self.params['b_proj']
        W_embed = self.params['W_embed']
        Wx, Wh, b = self.params['Wx'], self.params['Wh'], self.params['b']
        W_vocab, b_vocab = self.params['W_vocab'], self.params['b_vocab']
```

```
        ###########################################################################
        # TODO: Implement test-time sampling for the model. You will need to      #
        # initialize the hidden state of the RNN by applying the learned affine   #
        # transform to the input image features. The first word that you feed to  #
        # the RNN should be the <START> token; its value is stored in the         #
        # variable self._start. At each timestep you will need to do to:          #
        # (1) Embed the previous word using the learned word embeddings           #
        # (2) Make an RNN step using the previous hidden state and the embedded   #
        #     current word to get the next hidden state.                          #
        # (3) Apply the learned affine transformation to the next hidden state to #
        #     get scores for all words in the vocabulary                          #
        # (4) Select the word with the highest score as the next word, writing it #
        #     to the appropriate slot in the captions variable                    #
        #                                                                         #
        # For simplicity, you do not need to stop generating after an <END> token #
        # is sampled, but you can if you want to.                                 #
        #                                                                         #
        # HINT: You will not be able to use the rnn_forward or lstm_forward       #
        # functions; you'll need to call rnn_step_forward or lstm_step_forward in #
        # a loop.                                                                 #
        ###########################################################################
        # Perform image feature projection
        h0 = features @ W_proj + b_proj
        captions[:, 0] = self._start
        prev_h = h0
        prev_c = np.zeros_like(h0)
        # Current word (start word): (N, T)
        capt = self._start * np.ones((N, 1), dtype=np.int32)

        for t in range(max_length):
            word_embed, _ = word_embedding_forward(capt, W_embed)
            if self.cell_type == 'rnn':
              # Squeeze: (N, 1, D) -> (N, D)
              h, _ = rnn_step_forward(word_embed.squeeze(), prev_h, Wx, Wh, b)
            elif self.cell_type == 'lstm':
               h, c, _ = lstm_step_forward(word_embed.squeeze(), prev_h, prev_c, Wx, Wh, b)
            # Compute scores distrib over the dictionary. Mimic rnn classifier while using
            # temporal_affine layer by adding a time dimension, h:(N, H) -> h:(N, T, H),
            # where T = 1 using np.newaxis
            scores, _ = temporal_affine_forward(h[:, np.newaxis, :], W_vocab, b_vocab)
            # scores: (N, T, V) -argmax-> (N, T) -squeeze-> (N,), where T = 1
            idx_best = np.squeeze(np.argmax(scores, axis=2))
            captions[:, t] = idx_best

            # Update the hidden state, the cell state (if lstm) and the current word
            prev_h = h
            if self.cell_type == 'lstm':
                prev_c = c
            capt = captions[:, t]

        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
        return captions
```

2. Transformer decoder for image captioning:
    1. Transformer decoder layers for image captioning is available in Transformers.
    2. Code: classifiers/transformer.py

```
import numpy as np
import copy

import torch
import torch.nn as nn

from ..transformer_layers import *


class CaptioningTransformer(nn.Module):
    """
    A CaptioningTransformer produces captions from image features using a
    Transformer decoder.

    The Transformer receives input vectors of size D, has a vocab size of V,
    works on sequences of length T, uses word vectors of dimension W, and
    operates on minibatches of size N.
    """

    def __init__(
        self,
        word_to_idx,
        input_dim,
        wordvec_dim,
        num_heads=4,
        num_layers=2,
        max_length=50,
    ):
        """
        Construct a new CaptioningTransformer instance.
```

```
        Inputs:
        - word_to_idx: A dictionary giving the vocabulary. It contains V entries.
          and maps each string to a unique integer in the range [0, V).
        - input_dim: Dimension D of input image feature vectors.
        - wordvec_dim: Dimension W of word vectors.
        - num_heads: Number of attention heads.
        - num_layers: Number of transformer layers.
        - max_length: Max possible sequence length.
        """
        super().__init__()

        vocab_size = len(word_to_idx)
        self.vocab_size = vocab_size
        self._null = word_to_idx["<NULL>"]
        self._start = word_to_idx.get("<START>", None)
        self._end = word_to_idx.get("<END>", None)

        self.visual_projection = nn.Linear(input_dim, wordvec_dim)
        self.embedding = nn.Embedding(vocab_size, wordvec_dim, padding_idx=self._null)
        self.positional_encoding = PositionalEncoding(wordvec_dim, max_len=max_length)

        decoder_layer = TransformerDecoderLayer(
            input_dim=wordvec_dim, num_heads=num_heads
        )
        self.transformer = TransformerDecoder(decoder_layer, num_layers=num_layers)
        self.apply(self._init_weights)

        self.output = nn.Linear(wordvec_dim, vocab_size)

    def _init_weights(self, module):
        """
        Initialize the weights of the network.
        """
        if isinstance(module, (nn.Linear, nn.Embedding)):
            module.weight.data.normal_(mean=0.0, std=0.02)
            if isinstance(module, nn.Linear) and module.bias is not None:
                module.bias.data.zero_()
        elif isinstance(module, nn.LayerNorm):
            module.bias.data.zero_()
            module.weight.data.fill_(1.0)

    def forward(self, features, captions):
        """
        Given image features and caption tokens, return a distribution over the
        possible tokens for each timestep. Note that since the entire sequence
        of captions is provided all at once, we mask out future timesteps.

        Inputs:
         - features: image features, of shape (N, D)
         - captions: ground truth captions, of shape (N, T)

        Returns:
         - scores: score for each token at each timestep, of shape (N, T, V)
        """
        N, T = captions.shape
        # Create a placeholder, to be overwritten by your code below.
        scores = torch.empty((N, T, self.vocab_size))
        ############################################################################
        # TODO: Implement the forward function for CaptionTransformer.             #
        # A few hints:                                                             #
        #  1) You first have to embed your caption and add positional              #
        #     encoding. You then have to project the image features into the same  #
        #     dimensions.                                                          #
        #  2) You have to prepare a mask (tgt_mask) for masking out the future     #
        #     timesteps in captions. torch.tril() function might help in preparing #
        #     this mask.                                                           #
        #  3) Finally, apply the decoder features on the text & image embeddings   #
        #     along with the tgt_mask. Project the output to scores per token      #
        ############################################################################
        # input_dim: D
        # wordvec_dim: W
        # vocab_dim: V
        # captions: (N, T) with elements in the range: 0 to V-1 i.e. range(0, V)
        # features: (N, D)
        # w_embed: (V, W)
        x = self.embedding(captions)  # (N, T, W)
        x = self.positional_encoding(x)  # (N, T, W)
        _, _, W = x.shape
        # (N, D)->(N, W)->(N, 1, W)->(N, T, W)
        h = self.visual_projection(features).unsqueeze(1).expand(N, T, W)
        tgt_mask = torch.ones(T, T)  # (T, T)
        tgt_mask = torch.tril(tgt_mask) > 0
        out = self.transformer(x, h, tgt_mask)  # (N, T, W)
        scores = self.output(out)  # (N, T, V)

        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################
```

```
        return scores

    def sample(self, features, max_length=30):
        """
        Given image features, use greedy decoding to predict the image caption.

        Inputs:
         - features: image features, of shape (N, D)
         - max_length: maximum possible caption length

        Returns:
         - captions: captions for each example, of shape (N, max_length)
        """
        with torch.no_grad():
            features = torch.Tensor(features)
            N = features.shape[0]

            # Create an empty captions tensor (where all tokens are NULL).
            captions = self._null * np.ones((N, max_length), dtype=np.int32)

            # Create a partial caption, with only the start token.
            partial_caption = self._start * np.ones(N, dtype=np.int32)
            partial_caption = torch.LongTensor(partial_caption)
            # [N] -> [N, 1]
            partial_caption = partial_caption.unsqueeze(1)

            for t in range(max_length):

                # Predict the next token (ignoring all other time steps).
                output_logits = self.forward(features, partial_caption)
                output_logits = output_logits[:, -1, :]

                # Choose the most likely word ID from the vocabulary.
                # [N, V] -> [N]
                word = torch.argmax(output_logits, axis=1)

                # Update our overall caption and our current partial caption.
                captions[:, t] = word.numpy()
                word = word.unsqueeze(1)
                partial_caption = torch.cat([partial_caption, word], dim=1)

            return captions


def clones(module, N):
    "Produce N identical layers."
    return nn.ModuleList([copy.deepcopy(module) for _ in range(N)])


class TransformerDecoder(nn.Module):
    def __init__(self, decoder_layer, num_layers):
        super().__init__()
        self.layers = clones(decoder_layer, num_layers)
        self.num_layers = num_layers

    def forward(self, tgt, memory, tgt_mask=None):
        output = tgt

        for mod in self.layers:
            output = mod(output, memory, tgt_mask=tgt_mask)

        return output


class TransformerEncoder(nn.Module):
    def __init__(self, encoder_layer, num_layers):
        super().__init__()
        self.layers = clones(encoder_layer, num_layers)
        self.num_layers = num_layers

    def forward(self, src, src_mask=None):
        output = src

        for mod in self.layers:
            output = mod(output, src_mask=src_mask)

        return output
```

3. from show, attend and tell [4]:
    1. Extract a 14x14 Feature Map from CNN.
    2. Image: $H \times W \times 3$ -CNN-> Features, v: $L \times D$, $L = W \times H$, this Feature vector becomes an input to become an RNN's first hidden state h_0. The RNN outputs a distribution over L possible locations, a_1. These are the locations of these grids in the feature vector $L \times D$. a_1 is multiplied with the feature map to get a weighted features $z_1 = \sum_{i=1}^{L} a_i v_i$ (v_i are the features inside each grid locations that come as the output of the feature map) which will become input together with y_1 (first word) or the start token (SOS) in the next RNN time step.

3. h_1 which is the the next hidden step of the next time step in the RNN, gives us a_2 and d_1, a distribution over vocabulary that tells us what is the predicted word. a_2 is again multiplied with the feature map to give us a new weighted representation of the image feature map z_2. And d_1 could be the input y_2, or when we're training we're training we could use the second word of the caption to be input y_2 to the next step of the RNN, so z_2 and y_2 are the inputs to hidden state to the next step in the RNN. This generates a_3 and d_2. d_2 is the distribution over the vocabulary which tells us what is the word, a_3 tells us which part of the image to focus on. This is repeated again and again, and the caption is generated.
4. In soft attention the a_1 the weight vector, is a softmax vector, and we use it as weights to multiply with each of grid locations in v. This gives us a soft attention or the feature map of the entire image.
5. In hard attention we only choose a winning entry and only use that grid location as an input to the next time step. Hard since we're not using a soft distribution of the weights across all parts of the image.

4. CLIP
    1. Motivation:
        1. Rather than training models with fixed set of predetermined object categories (in image classification) that limits their flexibility. Can we train a vision model to work "zero-shot"?
        2. Prior works in NLP have shown that learning from descriptions rather than fixed labels can be very data efficient. For example VirTex demonstrated data efficiency of captioning. ConVIRT showed data efficiency of contrastive training.
        3. Scale: NLP systems have benefited tremendously from scale. GPT-3 showed zero-shot transfer scale benefits.
    2. CLIP learns from text-image pairs that are already publicly available on the internet using self-supervised learning, contrastive methods, self-training approaches, and generative modeling.
    3. First key idea: use a text encoder as a classifier [5]
    4. This is an old idea -- words and pictures work goes back to ~2000, but at a smaller scale.
    5. How to scale?
        1. Learn from natural language supervision (not tags or class labels)
        2. Scrape 400 million image/text pairs
        3. "bag of words" language representation
        4. Contrastive objective, instead of predicting exact language
        5. Use transformer architecture
    6. Use small transformer language model (76M parameters for base)
    7. Matching task with large batch (size=32768)
        1. Each image and text from batch is encoded
        2. Similarity score obtained for 32k x 32k image-text pairings
        3. Loss is cross-entropy on matching each image to its text, and each text to its image

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of alighed images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of iamge to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I)  #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t) / 2

"""
Numpy-like pseudocode for the core of an implementation of CLIP.
"""
```

        8. Related: Pointwise Mutual Information (PMI) solver [6].
            1. The PMI solver applies associational knowledge. Given a question q and an answer option a_i, It uses pointwise mutual information (Church and Hanks 1989) to measure the strength of the associations between parts of q and

parts of a_i. Given a large corpus C, PMI for two n-grams x and y is defined as: $PMI(x, y) = \log \frac{p(x,y)}{p(x)p(y)}$

References

1. Paper: An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition - Baoguang Shi et al
2. Convolutional Image Captioning, Jyoti Aneja, Aditya Deshpande, Alexander G. Schwig.
3. Karpathy et al, Deep visual-semantic alignments for generating image description, CVPR 2015.
4. Xu et al, Show attend and tell: Neural Image Caption generation with Visual Attention, ICML 2015.
5. CS441, Applied Machine Learning, Derek Hoiem, 2023.
6. Clark, P., Etzioni, O., Khot, T., Sabharwal, A., Tafjord, O., Turney, P., & Khashabi, D. (2016). Combining Retrieval, Statistics, and Inference to Answer Elementary Science Questions. *Proceedings of the AAAI Conference on Artificial Intelligence*, *30*(1). https://doi.org/10.1609/aaai.v30i1.10325